



ORGANIZACIÓN DEL COMPUTADOR

Curso Benítez

Trabajo Práctico

Ingeniería Inversa en Runtime con ptrace

XX

7 de junio de 2026

Agliani, Ignacio

114276

Índice

1. Introducción	3
2. Conceptos fundamentales	3
2.1. Formato ELF y ejecutables PIE	3
2.2. La llamada al sistema <code>ptrace</code>	4
2.3. Breakpoints de software	4
3. Análisis del ejecutable	5
3.1. Información general	5
3.2. Flujo de ejecución	5
3.3. Las seis fases	5
3.4. Detalle de las comparaciones	6
4. Implementación del desactivador	6
4.1. Estrategia de inyección	6
4.2. Arquitectura del programa	7
4.3. Detección de la dirección base	7
4.4. Gestión de breakpoints	8
4.5. Lectura y escritura de 32 bits en la memoria del tracee	8
4.6. El loop principal y manejo de señales	9
4.7. Inyección por tipo de fase	9
5. Compilación y ejecución	10
6. Resultados	10
7. Conclusión	11

1. Introducción

El *BombLab* es un ejercicio clásico de ingeniería inversa propuesto originalmente por Bryant y O'Hallaron en la Universidad Carnegie Mellon. En su versión tradicional, un ejecutable solicita seis claves al usuario; si alguna es incorrecta, la “bomba explota” y el proceso termina con error. Como las claves están codificadas en el binario, es posible encontrarlas mediante análisis estático del código ensamblador.

La variante presentada en este trabajo introduce una diferencia fundamental: **las claves no están almacenadas en el ejecutable sino que se generan aleatoriamente en cada ejecución**, utilizando la función `rand()` inicializada con `srand(time(NULL))`. Esto hace que el análisis estático sea insuficiente: no existe ninguna clave fija que pueda extraerse del binario con antelación.

El objetivo del trabajo es escribir un programa en C que:

1. Ejecute el binario *Bomba* como proceso hijo.
2. Utilice la llamada al sistema `ptrace` para interceptar su ejecución en tiempo real.
3. Lea el valor esperado en cada fase directamente desde los registros o la memoria del proceso.
4. Inyecte ese valor en el operando que representa la respuesta del usuario, justo antes de que se evalúe la comparación.
5. Logre pasar las seis fases sin que la bomba detone.

Este informe detalla los conceptos teóricos involucrados, el análisis del ejecutable mediante ingeniería inversa y el diseño del programa `desactivador.c`.

2. Conceptos fundamentales

2.1. Formato ELF y ejecutables PIE

Los programas en Linux se almacenan en formato **ELF** (*Executable and Linkable Format*). Un archivo ELF contiene, entre otras cosas, *segmentos de carga* que el sistema operativo mapea en memoria virtual cuando ejecuta el programa. Cada segmento tiene una dirección virtual de carga definida en el encabezado del ELF.

Un ejecutable **PIE** (*Position Independent Executable*) puede ser cargado en cualquier dirección de memoria, ya que todas sus instrucciones usan referencias relativas al *instruction pointer* (`rip`) en lugar de direcciones absolutas. Las direcciones que aparecen en el desensamblado son *offsets* relativos a la base, no direcciones reales en memoria.

El binario *Bomba* es un ELF64 PIE. Una de sus instrucciones en el desensamblado aparece en `0x176e`, pero en tiempo de ejecución estará en alguna dirección como `0x584ae0e5876e`, dependiendo de dónde el sistema operativo lo haya cargado.

ASLR — Address Space Layout Randomization

El kernel de Linux implementa **ASLR**, que asigna una dirección base aleatoria a cada proceso en cada ejecución. Combinado con PIE, esto significa que la dirección de cualquier instrucción cambia en cada ejecución. Por lo tanto, para colocar un breakpoint es necesario:

1. Conocer el *offset* de la instrucción en el ELF (obtenido del desensamblado).
2. Determinar la dirección base real en runtime.
3. Calcular: `dir_real = base_carga + offset_ELF`.

La dirección base se obtiene leyendo el archivo `/proc/<pid>/maps`, que lista todos los segmentos mapeados del proceso.

2.2. La llamada al sistema ptrace

`ptrace` (*process trace*) es una llamada al sistema de Linux que permite a un proceso *trazador* (*tracer*) observar y controlar la ejecución de otro proceso *trazado* (*tracee*). Es la base sobre la que se construyen los depuradores como GDB y LLDB.

El mecanismo general es el siguiente: cuando el tracee recibe una señal (incluido `SIGTRAP`, generado por los breakpoints), se suspende y el tracer es notificado vía `waitpid()`. El tracer puede entonces leer y modificar el estado completo del tracee (registros y memoria) antes de reanudarlo.

Las operaciones de `ptrace` utilizadas en este trabajo son:

Operación	Descripción
<code>PTRACE_TRACEME</code>	El proceso hijo se declara trazable por su padre. Debe llamarse antes de <code>execve</code> .
<code>PTRACE_CONT</code>	Reanuda la ejecución del tracee. Opcionalmente entrega una señal al proceso.
<code>PTRACE_PEEKTEXT</code>	Lee una palabra de 8 bytes de la memoria virtual del tracee.
<code>PTRACE_POKETEXT</code>	Escribe una palabra de 8 bytes en la memoria virtual del tracee.
<code>PTRACE_GETREGS</code>	Copia todos los registros del procesador (<code>rip</code> , <code>rsp</code> , <code>rax</code> , <code>rbx</code> , ..., <code>r15</code>) a una estructura <code>user_regs_struct</code> en el tracer.
<code>PTRACE_SETREGS</code>	Escribe valores en los registros del tracee a partir de una estructura <code>user_regs_struct</code> .

2.3. Breakpoints de software

Un **breakpoint de software** consiste en reemplazar el primer byte de una instrucción con `0xCC`, que corresponde a la instrucción `INT3` (*breakpoint interrupt*) del x86. Cuando el procesador ejecuta `INT3`, genera automáticamente una excepción que el sistema operativo convierte en una señal `SIGTRAP` para el proceso.

El procedimiento completo para colocar y usar un breakpoint es:

1. **Leer** el byte original en la dirección objetivo (con `PTRACE_PEEKTEXT`).
2. **Guardarlo** para poder restaurarlo después.
3. **Escribir** `0xCC` en esa dirección (con `PTRACE_POKETEXT`).
4. **Reanudar** el tracee con `PTRACE_CONT`.
5. Cuando el breakpoint se dispara, `waitpid` retorna con señal `SIGTRAP`.
6. **Notar** que `rip` apunta al byte *siguiente* al `0xCC` (el CPU ya incrementó el PC al ejecutar `INT3`).
7. **Restaurar** el byte original con `PTRACE_POKETEXT`.
8. **Corregir** `rip` decrementándolo en 1 con `PTRACE_SETREGS`, para que apunte de nuevo a la instrucción original.
9. **Realizar** las modificaciones deseadas (registros, memoria).
10. **Continuar** con `PTRACE_CONT`.

3. Análisis del ejecutable

3.1. Información general

```
$ file Bomba
Bomba: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
       dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
       BuildID[sha1]=2ec7e50a..., for GNU/Linux 3.2.0, stripped
```

Características relevantes del binario:

- **Arquitectura:** x86-64 (64 bits, little-endian).
- **Tipo:** ELF PIE, enlazado dinámicamente.
- **Símbolos:** *stripped* — no contiene información de debug ni nombres de funciones.
- **Librerías:** libc estándar. Las funciones importadas visibles en el PLT son: `rand`, `srand`, `time`, `fgets`, `sscanf`, `strcmp`, `strcspn`, `puts`, `printf`, `exit`.

El análisis se realizó con `objdump -d Bomba`, cuyo resultado completo se encuentra en `desensamblado.txt`.

3.2. Flujo de ejecución

Al iniciarse, la función `main` del binario sigue este patrón:

1. Llama a `time(NULL)` para obtener la marca de tiempo actual en segundos.
2. Usa ese valor como semilla con `srand()`.
3. Realiza 28 llamadas a `rand()` y aplica operaciones aritméticas (XOR, sumas, módulos) para derivar las claves de las seis fases. Todas quedan almacenadas en registros o en el stack antes de mostrar el primer prompt.
4. Para cada fase, en orden: imprime un prompt, lee la respuesta del usuario con `fgets` + `sscanf` (o solo `fgets` + `strcmp` para la fase de texto), compara con la clave esperada y, si no coincide, salta a la función de detonación que imprime `BOOM!` y termina con `exit(1)`.

3.3. Las seis fases

El análisis del desensamblado permitió identificar las instrucciones de comparación de cada fase, los offsets desde la base PIE y qué registro o posición de memoria contiene el valor esperado:

Fase	Tipo	Offset ELF	Instrucción de comparación	Clave esperada en
1	Entero	0x176e	<code>cmp [rsp+0x44], r15d</code>	registro <code>r15d</code>
2	Float	0x17c8	<code>ucomiss xmm3, [rsp+0x48]</code>	memoria <code>[rsp+0x1c]</code>
3	Entero	0x182f	<code>cmp [rsp+0x44], ebx</code>	registro <code>ebx</code>
4	Texto	0x18ae	<code>call strcmp(rdi, rsi)</code>	<code>rsi</code> → string esperado
5	Entero	0x1911	<code>cmp [rsp+0x44], ebx</code>	registro <code>ebx</code>
6	Float	0x195f	<code>ucomiss xmm4, [rsp+0x4c]</code>	memoria <code>[rsp+0x3c]</code>

En todos los casos, la respuesta del usuario es leída por `sscanf` (formatos `%d` para enteros y `%f` para floats) y almacenada en `[rsp+0x44]` (enteros) o `[rsp+0x48]`/`[rsp+0x4c]` (floats). La bomba compara ese valor con la clave esperada.

3.4. Detalle de las comparaciones

Fases 1, 3 y 5 — Enteros

La estructura es idéntica en los tres casos: el usuario debe ingresar el valor exacto que está almacenado en un registro. En la fase 1 ese registro es `r15d`; en las fases 3 y 5 es `ebx`, aunque con distinto valor en cada caso ya que se calcula en distintos momentos de la ejecución.

```
1 176e: 44 39 7c 24 44    cmp    DWORD PTR [rsp+0x44], r15d
2 1773: 0f 85 22 02 00    jne    199b      ; saltar a BOOM! si no coincide
```

Listing 1: Comparación de entero — Fase 1 (offset 0x176e)

Fases 2 y 6 — Punto flotante

Los floats se comparan con la instrucción SSE `ucomiss`, que compara dos valores en precisión simple (float de 32 bits). El valor esperado se carga en un registro XMM desde el stack y se compara con el input del usuario.

```
1 17c2: f3 0f 10 5c 24 1c  movss  xmm3, DWORD PTR [rsp+0x1c] ; carga
   clave
2 17c8: 0f 2e 5c 24 48    ucomiss xmm3, DWORD PTR [rsp+0x48] ; compara
   con input
3 17cd: 0f 8a c8 01 00 00  jp      199b      ; BOOM si NaN (parity flag)
4 17d3: 0f 85 c2 01 00 00  jne     199b      ; BOOM si no son iguales
```

Listing 2: Comparación de float — Fase 2 (offset 0x17c2)

La instrucción `ucomiss` activa el *parity flag* si alguno de los operandos es NaN. Las dos ramas condicionales (`jp` y `jne`) garantizan que tanto NaN como valores distintos detonen la bomba.

Fase 4 — Texto

La bomba construye un string de 15 caracteres ASCII imprimibles (generados con `rand() % 93 + 33`) y lo almacena en `[rsp+0x50]`. El usuario debe ingresar exactamente ese string. La comparación se hace con `strcmp`.

```
1 18a1: 48 8d 74 24 50    lea    rsi, [rsp+0x50]          ; rsi = string esperado
2 18a6: 48 89 ef          mov    rdi, rbp                ; rdi = buffer del
   usuario
3 18a9: c6 44 04 60 00    mov    BYTE PTR [rsp+rax+0x60], 0x0 ; null-terminar
   input
4 18ae: e8 8d f8 ff ff    call   strcmp                  ; compara
5 18b3: 85 c0            test   eax, eax
6 18b5: 0f 85 e0 00 00    jne    199b                    ; BOOM si != 0
```

Listing 3: Comparación de string — Fase 4 (offset 0x18ae)

El string esperado ocupa 15 bytes más un terminador nulo en `[rsp+0x5f]`.

4. Implementación del desactivador

4.1. Estrategia de inyección

Existen dos enfoques posibles para resolver el problema:

Enfoque A — Replicar la aritmética: Capturar la semilla de `srand` con `ptrace` y reproducir en el proceso padre todas las operaciones del binario (28 llamadas a `rand()` seguidas de operaciones de XOR, suma y módulo) para calcular las 6 claves de antemano. Se escriben las respuestas correctas en el pipe antes de que el hijo las pida.

Ventaja: una sola lectura, sin necesidad de múltiples breakpoints. *Desventaja:* requiere re-implementar exactamente la aritmética de división entera con “números mágicos” que usa el compilador, y las conversiones de punto flotante. Cualquier error de precisión hace fallar la fase correspondiente.

Enfoque B — Inyección en el punto de comparación: Colocar breakpoints en las instrucciones de comparación. En ese momento la clave esperada ya está calculada y disponible en registros o en el stack. Se sobrescribe el operando que representa la respuesta del usuario con el valor correcto.

Ventaja: no depende de ningún detalle aritmético del binario ni de la semilla. *Desventaja:* requiere 6 breakpoints y enviar respuestas ficticias en el pipe.

Se optó por el **Enfoque B** por ser más robusto y menos propenso a errores. Las respuestas ficticias enviadas al pipe ("0", "0.0", etc.) son leídas por `fgets` pero sus valores son irrelevantes: `ptrace` los sobrescribe antes de que se ejecute la comparación.

4.2. Arquitectura del programa

El flujo general de `desactivador.c` es el siguiente:

1. Crear un pipe (extremo de escritura para el padre, extremo de lectura como `stdin` del hijo).
2. Hacer `fork()`. El hijo: conecta `stdin` al pipe, llama a `PTRACE_TRACEME` y ejecuta `execve("./Bomba")`.
3. El padre espera el `SIGTRAP` inicial que `execve` genera en un proceso trazado.
4. El padre lee `/proc/<pid>/maps` para obtener la dirección base del ELF.
5. El padre coloca 6 breakpoints y escribe 6 respuestas ficticias en el pipe.
6. El padre entra en un loop que procesa cada breakpoint en orden:
 - a) Reanuda al hijo con `PTRACE_CONT`.
 - b) Espera `SIGTRAP` (otras señales se reenvían al hijo).
 - c) Lee registros con `PTRACE_GETREGS`.
 - d) Inyecta el valor correcto según el tipo de fase.
 - e) Restaura la instrucción original y corrige `rip`.
 - f) Aplica los cambios con `PTRACE_SETREGS`.
7. Tras los 6 breakpoints, reanuda al hijo hasta que termina.

4.3. Detección de la dirección base

```
1 static unsigned long get_base(pid_t pid)
2 {
3     char path[64];
4     snprintf(path, sizeof(path), "/proc/%d/maps", (int)pid);
5     FILE *f = fopen(path, "r");
6
7     unsigned long base = 0;
8     char line[512];
9     while (fgets(line, sizeof(line), f)) {
10         if (strstr(line, "Bomba")) { /* buscar la entrada del binario */
11             sscanf(line, "%lx-", &base);
12             break;
13         }
14     }
```

```

15     fclose(f);
16     return base;
17 }

```

Listing 4: Función get_base: dirección base del ELF PIE desde /proc/pid/maps

La primera entrada en /proc/<pid>/maps que corresponde al binario Bomba indica la dirección donde fue cargado su primer segmento (el que corresponde a la dirección virtual 0x0 en el ELF). Sumando un offset del desensamblado a esta base se obtiene la dirección real de cualquier instrucción en el proceso.

4.4. Gestión de breakpoints

```

1 static long set_bp(pid_t pid, unsigned long addr)
2 {
3     long orig = ptrace(PTRACE_PEEKTEXT, pid, (void *)addr, NULL);
4     /* Reemplazar solo el primer byte con 0xCC, preservar los 7 restantes */
5     long patched = (orig & ~0xFFL) | 0xCCL;
6     ptrace(PTRACE_POKETEXT, pid, (void *)addr, (void *)patched);
7     return orig; /* devolver para restaurar luego */
8 }
9
10 static void rm_bp(pid_t pid, unsigned long addr, long orig)
11 {
12     ptrace(PTRACE_POKETEXT, pid, (void *)addr, (void *)orig);
13 }

```

Listing 5: Colocación y remoción de breakpoints de software

PTRACE_POKETEXT opera sobre palabras de 8 bytes. Para no corromper los 7 bytes restantes de la instrucción, se aplica una máscara & ~0xFFL que preserva todos los bytes excepto el primero, y luego se OR con 0xCCL para insertar el breakpoint.

4.5. Lectura y escritura de 32 bits en la memoria del tracee

Las claves de tipo entero y float ocupan 4 bytes (32 bits). Sin embargo, PTRACE_PEEKTEXT/POKETEXT siempre opera con palabras de 8 bytes alineadas. Las funciones peek32 y poke32 resuelven este desajuste:

```

1 static uint32_t peek32(pid_t pid, unsigned long addr)
2 {
3     unsigned long aligned = addr & ~7UL; /* alinear a 8 bytes */
4     long w = ptrace(PTRACE_PEEKTEXT, pid, (void *)aligned, NULL);
5     int shift = (int)((addr & 7) * 8); /* offset en bits dentro de la
6     palabra */
7     return (uint32_t)((uint64_t)w >> shift);
8 }
9
10 static void poke32(pid_t pid, unsigned long addr, uint32_t val)
11 {
12     unsigned long aligned = addr & ~7UL;
13     uint64_t w = (uint64_t)ptrace(PTRACE_PEEKTEXT, pid, (void *)aligned,
14     NULL);
15     int shift = (int)((addr & 7) * 8);
16     /* Limpiar los 4 bytes de destino y escribir el nuevo valor */
17     w = (w & ~(0xFFFFFFFFULL << shift)) | ((uint64_t)val << shift);
18     ptrace(PTRACE_POKETEXT, pid, (void *)aligned, (void *)w);
19 }

```

Listing 6: Funciones de acceso a 32 bits en memoria del tracee

Ejemplo: para acceder a `[rsp+0x1c]`, con `rsp` alineado a 8 bytes: la palabra de 8 bytes está en `rsp+0x18` y los 4 bytes del float comienzan en el byte 4 de esa palabra. Por lo tanto `shift = 32`, y la operación de desplazamiento extrae exactamente los 4 bytes deseados.

4.6. El loop principal y manejo de señales

```
1 for (int phase = 0; phase < 6; phase++) {
2     void *sig_to_deliver = NULL;
3     for (;;) {
4         ptrace(PTRACE_CONT, child, NULL, sig_to_deliver);
5         waitpid(child, &status, 0);
6         if (WIFEXITED(status)) { /* salida prematura */ return 1; }
7         if (WSTOPSIG(status) == SIGTRAP) break; /* nuestro breakpoint */
8         /* Otra señal (ej. del linker): reenviarla y seguir esperando */
9         sig_to_deliver = (void *) (long) WSTOPSIG(status);
10    }
11    ptrace(PTRACE_GETREGS, child, NULL, &regs);
12    unsigned long rip = regs.rip - 1; /* posición real del 0xCC */
13    unsigned long rsp = regs.rsp;
14    /* ... identificar breakpoint, inyectar valor, restaurar ... */
15    rm_bp(child, bp_addr[idx], bp_orig[idx]);
16    regs.rip = rip; /* corregir rip */
17    ptrace(PTRACE_SETREGS, child, NULL, &regs);
18 }
```

Listing 7: Loop de procesamiento de breakpoints con manejo de señales inesperadas

El loop interno garantiza que señales ajenas al programa (como las que puede enviar el linker dinámico durante la carga) no interrumpen el flujo: se reenvían al hijo con `PTRACE_CONT` y se continúa esperando el `SIGTRAP` propio.

4.7. Inyección por tipo de fase

```
1 switch (idx) {
2 case 0: /* Fase 1 -- entero: clave en r15d, input usuario en [rsp+0x44] */
3     poke32(child, rsp + 0x44, (uint32_t)regs.r15);
4     break;
5
6 case 1: /* Fase 2 -- float: clave en [rsp+0x1c], input usuario en [rsp+0x48] */
7     poke32(child, rsp + 0x48, peek32(child, rsp + 0x1c));
8     break;
9
10 case 2: /* Fase 3 -- entero: clave en ebx, input usuario en [rsp+0x44] */
11     poke32(child, rsp + 0x44, (uint32_t)regs.rbx);
12     break;
13
14 case 3: /* Fase 4 -- texto: rdi = rsi -> strcmp compara el string consigo mismo */
15     regs.rdi = regs.rsi;
16     break;
17
18 case 4: /* Fase 5 -- entero: clave en ebx, input usuario en [rsp+0x44] */
19     poke32(child, rsp + 0x44, (uint32_t)regs.rbx);
20     break;
21
22 case 5: /* Fase 6 -- float: clave en [rsp+0x3c], input usuario en [rsp+0x4c] */
23     poke32(child, rsp + 0x4c, peek32(child, rsp + 0x3c));
24     break;
25 }
```

25 }

Listing 8: Lógica de inyección para cada una de las seis fases

La fase 4 merece una explicación especial. En lugar de copiar el string esperado al buffer del usuario, se modifica el registro `rdi` (primer argumento de `strcmp`) para que apunte a la misma dirección que `rsi` (segundo argumento, el string esperado). Así, `strcmp` compara el string esperado consigo mismo, retorna siempre 0, y la fase pasa sin necesidad de conocer ni copiar el contenido del string.

5. Compilación y ejecución

El programa se compila con GCC sin flags adicionales:

```
gcc -o desactivador desactivador.c
```

Y se ejecuta en el mismo directorio donde se encuentra el binario Bomba:

```
./desactivador
```

El ejecutable de la Bomba debe tener permisos de ejecución (`chmod +x Bomba`).

6. Resultados

La siguiente salida corresponde a una ejecución exitosa del desactivador en un sistema Linux x86-64 con kernel 6.x:

```
[*] Base de Bomba: 0x584ae0e57000
[*] BP fase 1 @ 0x584ae0e5876e
[*] BP fase 2 @ 0x584ae0e587c8
[*] BP fase 3 @ 0x584ae0e5882f
[*] BP fase 4 @ 0x584ae0e588ae
[*] BP fase 5 @ 0x584ae0e58911
[*] BP fase 6 @ 0x584ae0e5895f
[*] Desactivando bomba personalizada...

[*] === FASE 1 === Clave (int): 1664324801
[*] === FASE 2 === Clave (float): 282.568695 (bits=0x438d48cb)
[*] === FASE 3 === Clave (int): -903984316
[*] === FASE 4 === String esperado en rsi=0x7ffe2c9c13a0
[*] === FASE 5 === Clave (int): -727453326
[*] === FASE 6 === Clave (float): 626.644165 (bits=0x441ca93a)

[*] Todos los breakpoints procesados -- continuando...

Fase 1: Ingresa el codigo secreto (entero) >
Fase 2: Ingresa el codigo secreto (float) >
Fase 3: Ingresa el codigo secreto (entero) >
Fase 4: Ingresa el codigo secreto (texto) >
Fase 5: Ingresa el codigo secreto (entero) >
Fase 6: Ingresa el codigo secreto (float) >
Bomba desactivada con Exitó!

[*] Hijo termino con codigo 0
```

Listing 9: Salida completa de `./desactivador`

Los valores de las claves inyectadas en esa ejecución fueron:

Fase	Tipo	Valor inyectado
1	Entero	1664324801
2	Float	282.568695
3	Entero	-903984316
4	Texto	redirigido vía <code>rdi = rsi</code>
5	Entero	-727453326
6	Float	626.644165

Cabe destacar que en una ejecución posterior los valores cambian completamente (distinta semilla de `srand`), confirmando que las claves son efectivamente aleatorias y que el desactivador no depende de ningún valor fijo.

7. Conclusión

Se implementó exitosamente un programa en C que desactiva las seis fases de la bomba utilizando la llamada al sistema `ptrace`.

La estrategia de inyección en el punto de comparación resultó ser elegante y robusta: al actuar sobre los operandos en el preciso momento en que se evalúan, el desactivador no necesita conocer la aritmética interna del binario ni la semilla aleatoria de cada ejecución. El proceso de ingeniería inversa se centró únicamente en identificar *dónde* está cada comparación y *qué registro o posición de memoria* contiene el valor correcto, dejando toda la complejidad aritmética al propio binario.

El trabajo permitió poner en práctica de forma concreta conceptos fundamentales de arquitectura de computadoras y sistemas operativos: el formato ELF y el mecanismo PIE/ASLR, el funcionamiento a bajo nivel de `ptrace`, la implementación de breakpoints de software con la instrucción `INT3`, y la manipulación directa de registros y memoria de un proceso en ejecución.